

This push-based, agentless design is also why idempotency matters so much architecturally: since the control node re-evaluates and re-pushes on every run rather than a resident agent enforcing state continuously, each module is written to check current state first and only make changes when something is out of alignment.

Why Ansible?

Organisations often build large fleets of servers, containers, and cloud resources, and managing those fleets manually is slow and error prone. With Ansible, administrators can specify the desired state of a system in a simple human-readable file, and Ansible can make that state a reality on as many machines as the administrator desires.

Compared to alternatives such as Puppet, Chef, or SaltStack, Ansible is often preferred because it:

- Requires no agent software on managed nodes — only SSH and Python.
 - Uses YAML, a format that is easy to read even for people who are not full-time developers.
 - Has a very low learning curve, allowing teams to become productive within days.
 - Integrates cleanly with cloud providers, container platforms, and CI/CD pipelines.
 - Is backed by a large open source community and an extensive library of ready-made modules.
- Key features of Ansible are:

- *Agentless architecture*: Connects over SSH/WinRM; nothing to install or maintain on target hosts.
- *Idempotency*: Running the same playbook repeatedly produces the same end state without unwanted side effects.
- *Declarative playbooks*: YAML files describe the desired end state rather than step-by-step procedures.
- *Inventory management*: Hosts can be grouped logically (web servers, databases, etc) in static or dynamic inventories.

- *Extensive module library*: Has thousands of built-in modules covering package management, cloud provisioning, networking, and more.
- *Roles and reusability*: Playbooks can be organised into reusable, shareable roles via Ansible Galaxy.
- *Push-based execution*: The control node initiates all actions, giving predictable, on-demand automation.

Installing Ansible

Before installing Ansible, the following should be in place:

- A control node running Linux or macOS (Ansible does not run natively as a controller on Windows).
- Python 3.9 or later installed on the control node.
- SSH access (with key-based authentication recommended) from the control node to all managed nodes.
- Python installed on each managed node so Ansible modules can execute.
- Sudo/administrative privileges on managed nodes for tasks that require elevation.
- Basic familiarity with YAML syntax for writing playbooks.

On a Debian/Ubuntu-based control node, Ansible can be installed via the official PPA to ensure an up-to-date version:

```
sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository --yes --update ppa:ansible/ansible
sudo apt install -y ansible
```

On RHEL/CentOS/Fedora systems, the equivalent installation uses:

```
sudo dnf install -y ansible-core
```

Alternatively, Ansible can be installed in a Python virtual environment on any platform using pip:

```
python3 -m pip install --user ansible
```

```
user@ansible-control: ~
$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Reading package lists... Done
$ sudo apt install -y software-properties-common
Reading package lists... Done
software-properties-common is already the newest version.
$ sudo add-apt-repository --yes --update ppa:ansible/ansible
Repository: 'deb https://ppa.launchpadcontent.net/ansible/ansible/ubuntu/ noble main'
Adding repository.
Fetched 3,012 B in 1s
$ sudo apt install -y ansible
Reading package lists... Done
The following NEW packages will be installed:
  ansible ansible-core python3-jmespath python3-resolvelib
Setting up ansible-core (2.17.4-lppa-noble) ...
Setting up ansible (10.5.0-lppa-noble) ...
# Installation complete
```

Figure 2: Installing Ansible on Ubuntu via the official PPA

Configuring environment variables

Ansible's behaviour can be controlled through a configuration file (*ansible.cfg*) and a set of environment variables, which is useful for keeping project-specific settings out of the global system configuration. A typical project sets the following:

- *ANSIBLE_CONFIG* points to a project-specific *ansible.cfg* file.
- *ANSIBLE_INVENTORY* points to the inventory file listing managed hosts.
- *ANSIBLE_HOST_KEY_CHECKING* disables SSH host key prompts in lab/test environments (use with caution in production).